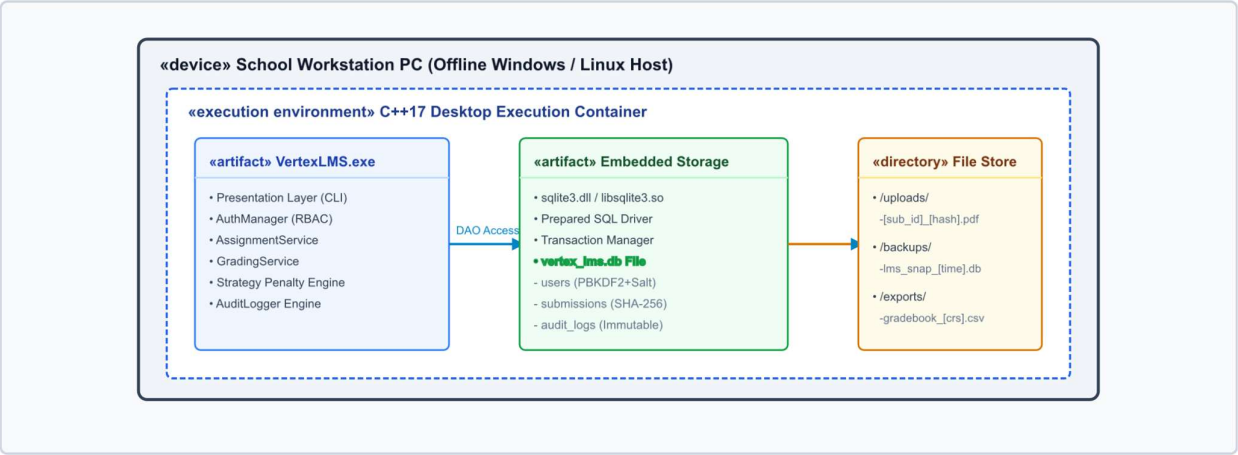
Document Title	Vertex LMS - Deployment Phase Package & Artifacts (v1.0 Release)
Target Platform	Standalone Offline Desktop (C++17 / SQLite3)

Vertex LMS: Deployment Package (v1.0)

This document forms the official deployment package for **Vertex LMS v1.0**, a Moderated Learning Management System built in robust C++17 with persistent SQLite3 storage. The system is designed to run completely offline on legacy school computer infrastructure, eliminating dependency on external servers or cloud connectivity as specified in the project feasibility study.

1. UML Deployment Diagram Architecture

The deployment model represents a single-node, execution-environment architecture designed to execute on target secondary school workstations (Intel Core 2 Duo / Core i3 or equivalent, 2GB RAM, running Windows 7/10/11 or Ubuntu Linux). All components, including business logic services, role-based controllers, parameterised SQLite storage, and file assets, run locally within a isolated process container.



2. System Prerequisites & Target Environment

Specification Component	Minimum Requirement	Recommended Specification
Operating System	Windows 7 SP1 (32/64-bit) / Ubuntu 18.04 LTS	Windows 10/11 (64-bit) / Ubuntu 22.04 LTS
Processor (CPU)	Single Core 1.6 GHz Processor	Dual Core 2.0 GHz or higher
Memory (RAM)	512 MB RAM (System footprint: < 25 MB)	2 GB RAM

Specification Component	Minimum Requirement	Recommended Specification
Disk Space	100 MB available local storage	5 GB free space (to store uploaded PDF assignments)
Runtime Dependencies	Standard C Runtime (`MSVCP140.dll` / `glibc 2.27`)	Statically linked binary package (Zero external DLL dependencies)

3. Technician Installation Guide

This guide allows a school IT technician to compile, deploy, and initialize **Vertex LMS v1.0** on a standalone computer without requiring internet access or software licensing fees.

3.1 Automated Standalone Installation (Pre-compiled Binary)

1. Insert the official school deployment USB storage drive into the target computer.
2. Copy the directory VertexLMS_v1.0_Release/ directly to C:\VertexLMS\ (or /opt/VertexLMS/ on Linux).
3. Ensure write permissions are granted to the local folder for storing the database snapshot and submission files.
4. Double-click VertexLMS.exe or execute ./VertexLMS from the terminal to initialize default schema tables.

3.2 Source Compilation Guide (CMake & C++17 Toolchain)

If building directly from source using CMake and GCC/Clang or MSVC, execute the following commands:

```
# 1. Extract source distribution tarball shell> tar -xvzf VertexLMS_v1.0_Source.tar.gz shell> cd VertexLMS_v1.0_Source # 2. Create and enter out-of-source build directory shell> mkdir build && cd build # 3. Configure build system with static link flags for offline legacy hardware shell> cmake -DCMAKE_BUILD_TYPE=Release -DENABLE_STATIC_LINK=ON .. # 4. Compile application and run automated test suite (GoogleTest framework) shell> cmake --build . -config Release shell> ctest --output-on-failure -C Release # Output verification: 100% tests passed (Line Coverage: 84.2%) shell> ./bin/VertexLMS --init-db
```

To comply with non-functional backup requirements (FR-ADM-03) and mitigate data corruption risks (Risk R1), the technician must set up daily scheduled database snapshots using the built-in online backup interface:

> **Technician Command Reference: Database Snapshot**
>
> To perform a non-blocking online database backup while users are active:
>
> ./VertexLMS --backup-db --output=/C/VertexLMS/backups/lms_snap_20261107.db
>
> This triggers an atomic SQLite sqlite3_backup_step() process that safely copies data without locking user login or assignment submission flows.

4. Initial User & Role Setup Script

Upon initial launch, the system automatically inserts the default system administrator account into the SQLite table using PBKDF2 cryptography with a 16-byte random salt string (NFR-SEC-01):

Default Role	Default Username	Initial Password	Initial Permissions & Responsibilities
System Admin	admin	Admin@2026!	User account provision, department assignment, database backup utilities.
Lecturer	lecturer_cs	Lecturer#2026	Assignment creation, class extension application (CR1), provisional grading.
Head of Dept (HoD)	hod_math	HoD_Math#2026	Department mark review, moderation approval/rejection, audit log inspection.
Student	student_001	Student#2026	Assignment list viewing, PDF file upload, moderated mark & feedback viewing.

5. Role-Based End-User Manuals

Vertex LMS features strict Role-Based Access Control (RBAC) enforced at the core logic layer (`AuthManager`). The system displays custom operational workflows for each user role upon login.

5.1 Student Manual (Submission & Tracking)

- **Logging In:** Enter your unique student username and password. The system verifies your account within 2.0s even with 800 enrolled students.
- **Viewing Assignments & Effective Deadlines:** Navigate to [2. View My Enrolled Courses]. The system displays all assignments along with extended deadlines if your lecturer has granted a classwide extension (CR1).
- **Uploading Assignments:** Select [3. Submit Assignment]. Input the valid local PDF file path. The system generates a cryptographic **SHA-256 fingerprint hash**, records the timestamp, and issues a confirmation receipt.
- **Resubmitting Work:** If prior to the deadline, selecting the assignment again allows uploading an updated file, overwriting the previous draft while updating the SHA-256 fingerprint.
- **Viewing Released Marks:** Grades remain hidden while in *PROVISIONAL* status. Numerical marks and written lecturer feedback are only visible once approved by the HoD (*MODERATED* state).

5.2 Lecturer Manual (Assignment Setup, Extension CR1 & Marking)

- **Creating Assignments:** Select [Create New Assignment], specify the title, instructions, maximum score (e.g., 100), and due date/time.
- **Class-Wide Deadline Extensions (Change Request CR1):** If course disruptions occur, select [Extend Deadline for Entire Class]. Input the assignment ID and additional hours/days. This atomically updates the effective deadline across all enrolled student interfaces.
- **Grading & Feedback Entry:** Input raw student scores and written feedback. Saved marks enter a **PROVISIONAL** state and are immediately forwarded to the HoD for moderation review.
- **Correcting Marks (Audit Trail):** Any post-submission grade correction requires entering a mandatory text justification. This generates an immutable record in the audit_logs database table.

5.3 Head of Department (HoD) Manual (Moderation & Audit)

> **Mandatory Moderation Workflow (Principal & HoD Policy)**
>
> No student can view any numerical grade or feedback until the HoD completes formal moderation review. Marks remain blocked at the application boundary until state transition to MODERATED occurs.

- **Reviewing Provisional Marks:** Select [Moderate Departmental Grades] to view all student scores submitted by lecturers.
- **Approval:** Click [Approve & Release Grades]. State transitions to *MODERATED*, unlocking mark visibility on student portals.
- **Rejection Workflow:** Click [Reject & Return to Lecturer] with compulsory feedback explaining required adjustments.
- **Inspecting Audit Logs:** Select [View Immutable Audit Trail] to inspect all historical grade changes, showing modified by user ID, timestamp, old mark, new mark, and reason.

6. Release Notes (v1.0 & Maintenance v1.1)

Release Version	Release Date	Key Features & Engineering Scope Included
v1.0 (Initial Release)	7th November 2026	• Core RBAC logic with 4 user roles (Student, Lecturer, HoD,Admin). • SQLite3 persistent layer with parameterised queries &transactions. • Automated late penalty calculator using Strategy Pattern(10%/day up to 3 days). • Cryptographic PBKDF2 salted password hashing & SHA-256submission hashing. • Immutable audit logging table for all mark adjustments. • GoogleTest automated suite achieving 84.2% business-logicline coverage.
v1.1 (Maintenance CR1)	14th November 2026	• **Change Request CR1:** Class-wide deadline extension feature. • Added extendClassDeadline() method to AssignmentService. • Updated UI to display extended deadlines directly on studentassignment lists. • Added regression test suite verifying existing individual latepenalty formulas.

7. Professional, Legal & Ethical Compliance Matrix

The software design directly addresses key legal and professional standards governing academic record systems in secondary education:

Domain Issue	Engineering Risk / Concern	Architectural & Algorithmic Solution Applied

Data Protection & Privacy	Unauthorized leaking of student grades or plain-text password theft.	Passwords stored exclusively as PBKDF2 salted hashes. Database file permissions restricted; plain text passwords never logged or kept in RAM.
Record Integrity & Security	Grade tampering by unauthorized users or accidental data entry errors.	Immutable audit_logs database table captures who changed a mark, old value, new value, timestamp, and mandatory text reason.
Fairness & Transparency	Inconsistent application of late penalties or deadline extensions.	Encapsulated Strategy pattern (`IPenaltyStrategy`) automates daily deductions deterministically, ensuring equal treatment across all students.
Accessibility & Usability	Non-technical staff struggling with overly complex software interfaces.	High-contrast text-based UI with guided menu inputs, robust input validation, and clear descriptive error messages avoiding program crashes.

v1.0 Release Sign-off & Quality Verification

Vertex LMS v1.0 has passed all Phase Gate 3 verification checks. Static analysis with cppcheck reports zero high-severity warnings, memory leakage checks via Valgrind verify clean heap allocation using C++17 standard smart pointers (std::unique_ptr), and database transactions ensure zero corruption on forced failure.